Курсовая работа

Ворушилов Константин Сергеевич

Введение

В работе исследовались методы разработки аппаратно-программного решения задачи Keyword Spotting (распознавание ключевого слова) в среде Arduino на отладочной плате Teensy 4.1.

Задача 1 *Разработать программно-аппаратное решение задачи Keyword Spotting на базе отладочной платы Teensy 4.1.*

Существует несколько методов решения подобной задачи, например, использование AutoML сервисов типа Edge Impulse [1]. Подобные сервисы обеспечивают высокую скорость и простоту работы. Тем не менее, разработка решений профессионального уровня с их использованием может быть ограничена из-за приватности тренировочных данных на стороне заказчика, эффекта «черного ящика» получаемого решения, необходимости более глубокого анализа получаемого решения и т.д.

В рамках работы была предпринята попытка пройти путь от разработки модели до встраивания в микроконтроллер с использованием экосистемы Arduino и Arduino-совместимых библиотек. В качестве целевой платформы был выбран микроконтроллер iMX.RT1062 на отладочной плате Teensy 4.1. Предобработка, тренировка, валидация, тестирование, квантование и конверсия были проведены с использованием пакетов PyTorch и Tensorflow.

В тестовых условиях модель продемонстрировала отличную способность к детектированию ключевого слова и низкую вероятность ложного срабатывания. Разработанный в рамках решения задачи пайплайн может быть использован для различных задач, связанных с обработкой и распознаванием аудиосигнала с использованием моделей глубокого обучения на микроконтроллере.

Сбор данных и обучение

Данные были собраны с помощью библиотеки Teensy Audio Library, входящей в состав пакета TeensyDuino для Arduino IDE. Данная библиотека, среди прочего, содержит пример прошивки, превращающей Teensy 4.1 в USB-аудиоустройство, способное осуществлять ввод и вывод аудиопотока. В качестве микрофона была использована отладочная плата на базе клона MEMS-микрофона ICS 43434. Подключение микрофона осушествлено по шине I^2S.

Всего было собрано около 15 минут данных, в их числе 3 минуты 30 секунд позитивных примеров (ключевое слово «Hi» , записанное с вариациями тембра голоса и относительной позиции голоса и микрофона) и около 10 минут негативных примеров (среди них слова, близкие к ключевым по произношению, разговорная речь как в исполнении целевого спикера, так и других людей, различные шумы, тишина), а также две отдельные записи для

[1] edgeimpulse.com

тестирования. Тренировочные данные разделены в соотношении 80:20 для проведения валидации между обучающими эпохами. Данные были сохранены в формате 8kHz, 16 bit PCM.

Предобработка данных была проведена с помощью пакета CMSIS-DSP от компании ARM. Функции из данного пакета имплементированы специально для микроконтроллеров ARM, в частности, ARM Cortex-M7, на основе которого выполняется данный проект. Пакет CMSIS-DSP для языка Python по сути представляет собой Python-обертку функций для микроконтроллера.

С помощью пакета CMSIS-DSP из аудиосигнала были извлечены мел-спектрограммы по следующей схеме:

1. Разделение сигнала на окна длиной 256 отсчетов с перекрытием 50%;
2. Применение окна Хэмминга для устранения краевых эффектов;
3. Получение коэффициентов Фурье для каждого окна;
4. Вычисление спектра мощности путем овеществления коэффициентов Фурье;
5. Отображение результатов на шкалу Mel [2].

Все вычисления проводились с помощью операций с фиксированной точкой (формат Q15) для совместимости с форматом данных PCM16. Нормализация сигнала не производилась; тем не менее, для согласования логики операций с фиксированной точкой между некоторыми операциями производились побитовые сдвиги. На вход модели подаются мел-спектрограммы с 30 последовательных окон, что примерно соответствует половине секунды при частоте дискретизации 8 кГц.

Архитектура модели глубокого обучения представляет собой сверточный классификатор, вдохновленный классической архитектурой LeNet [3]: три слоя свертка $\rightarrow$ активация $\rightarrow$ пуллинг, после чего данные выравниваются в одномерный вектор и передаются на двуслойный линейный классификатор. Результат классификатора пропускается через сигмоиду.

Для обучения модели использовался оптимизационный алгоритм Adam, оптимизирующий значения функции бинарной кросс-энтропии между выходами модели и значениями, кодирующими принадлежность сигнала классу (0 - негативный, не ключевое слово, 1 - позитивный, ключевое слово). Обучение проводилось в течение 100 эпох, чекпойнт каждой 10 эпохи сохранялся для последующего тестирования.

Квантование

Многие современные микроконтроллеры поддерживают операции с плавающей точкой, что, в теории, избавляет от необходимости квантовать модель перед портированием. Тем не менее, легковесные модели могут быть запущены на недорогих энергоэффективных микроконтроллерах, не обязательно поддерживающих вычисления с плавающей точкой. В работе было решено имплементировать модель, работающую с операциями только с фиксированной точкой.

Существует 2 основных подхода к квантованию нейронных сетей: QAT (quantisation-aware training, обучение с учетом квантования) и PTQ (post-training quantisation, квантование весов модели как есть после обучения). Оба метода, как правило, немного ухудшают

[2] https //en.wikipedia.org/wiki/Mel_scale
[3] https://en.wikipedia.org/wiki/LeNet

эффективность модели. При этом QAT известен тем, что модель, полученная этим методом, часто дает сравнимые или незначительно худшие результаты с изначальной моделью, в то время как PTQ приводит к заметной деградации модели. Ничего из этого не является универсальным правилом, поэтому зависимость результатов от использования QAT или PTQ необходимо исследовать отдельно.

В рамках работы над данной моделью было проведено сравнение QAT и PTQ. Изначально модель получалась методом PTQ из модели, обученой на Pytorch и импортированной во фреймворк Tensorflow Lite через ONNX. Метрики модели в процессе портирования не менялись, однако в процессе PTQ модель ощутимо деградировала; так метрика ROC-AUC показывала результат 0.6 против 0.92 у изначальной модели. Чтобы предотвратить сильную деградацию, тренировочный код Pytorch был переписан на Tensorflow с помощью Github Copilot, и в тренировочный процесс был внедрен QAT. Это позволило наблюдать гораздо меньший уровень деградации (ROC-AUC около 0.8 против 0.92 у модели до квантования).

Тестирование

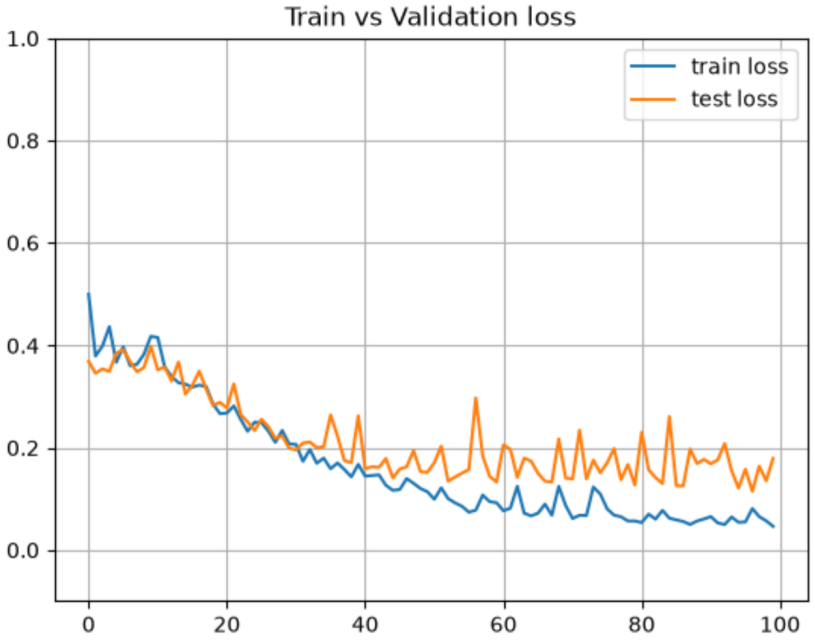


Рис. 1: Значения лосс-функции на тренировочной и валидационной выборках в течение обучения.

Если посмотреть на график значений лосс-функции на тренировочной и валидационной выборках в зависимости от эпохи (рис. 1), можно заметить, что, начиная с примерно 30 эпохи, лосс-функция на валидационном наборе данных дает более высокие разультаты. Обычно такая ситуация свидетельствует о переобучении модели на тренировочных данных. В нашем случае было замечено, что, несмотря на просадку валидационного лосса, на тестовом наборе чекпойнты, полученные на поздних стадиях обучения, показывали результаты лучше, чем чекпойнты ранних стадий. На рис. 2 и рис. 3 показано сравнение кривых AUC и Precision-Recall для эпох 30 и 100; видно, что модель показывает немного лучший перформанс на позднем чекпойнте.

В качестве порогового значения классификатора было выбрано достаточно консервативное значение 0.9 с целью уменьшения ложноположительных детекций.

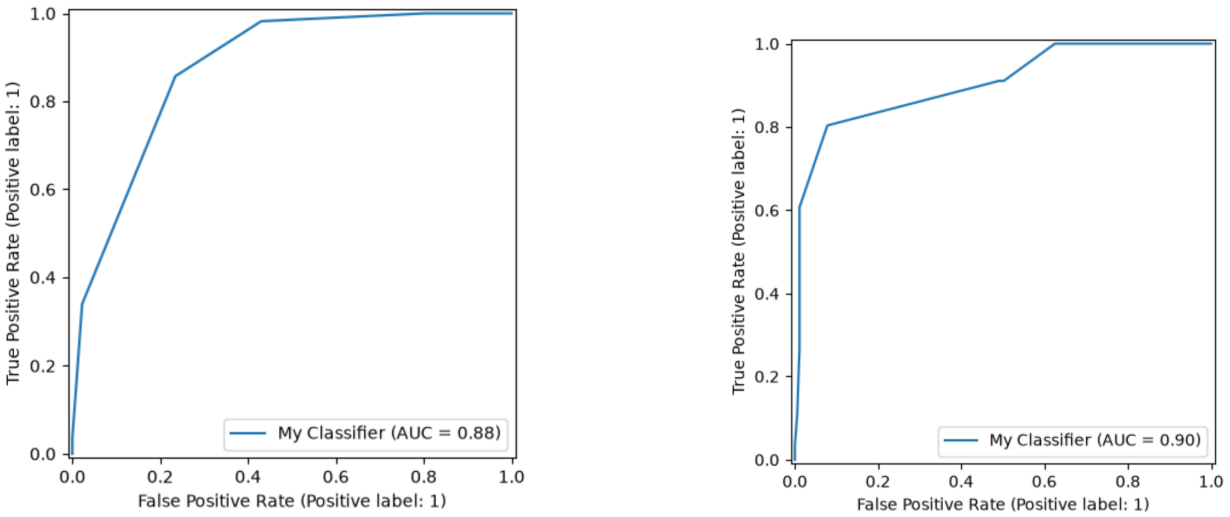


Рис. 2: Сравнение ROC-кривых, слева - чекпойнт эпохи 30, справа - чекпойнт эпохи 100.

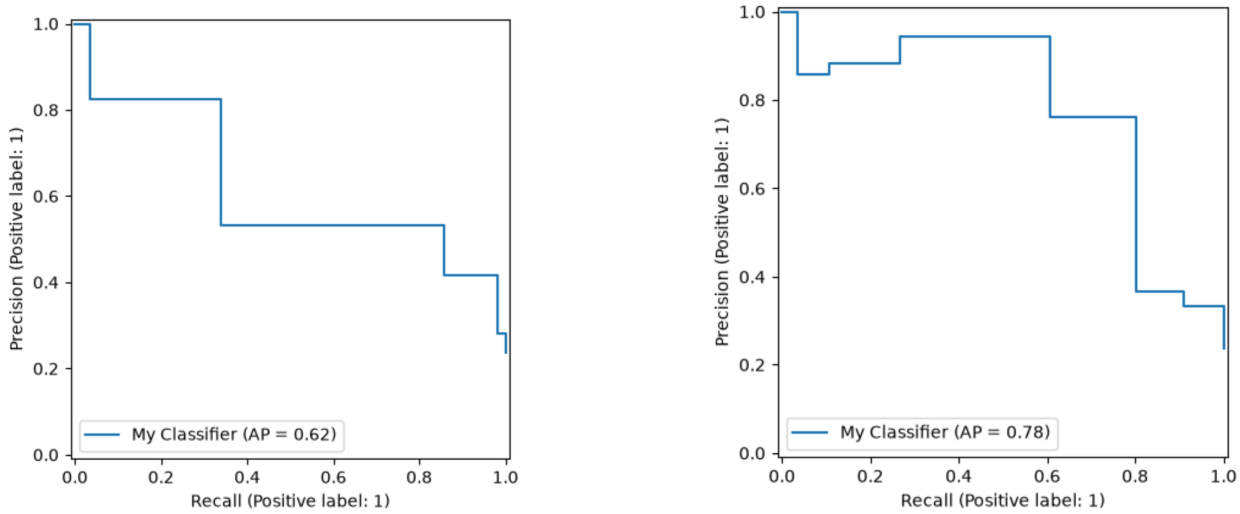


Рис. 3: Сравнение кривых Precision-Recall, слева - чекпойнт эпохи 30, справа - чекпойнт эпохи 100.

Прошивка

Для запуска модели на микроконтроллере был использован фреймворк MicroTFLite от johnosbb [4]. Данный фреймворк поддерживает широкий ряд отладочных плат Arduino, и, несмотря на отсутствие официальной поддержки других плат, полностью работоспособен на плате Teensy 4.1. Для простоты работы с данными в формате с фиксированной точкой, функции ввода исходных данных и вывода полученного результата инференса были слегка видоизменены для данного проекта и размещены в файлах `custom_model_io.h/.cpp`. Параметры модели были сконвертированы из .tflite-файла в формат массива языка Си с помощью утилиты xxd. Их вес составляет примерно 10 килобайт (10352 байта).

Для получения данных с I^2S-MEMS микрофона была использована библиотека Teensy Audio Library. К сожалению, библиотека ограничена в режиме работы форматом 44100 kHz 16-bit, из-за чего для получения данных вручную был имплементирован даунсэмплинг до 8 kHz с применением реконструкционного симметричного FIR-фильтра (сгенерированного с помозью scipy.firwin). Процедура даунсэмплинга понижает частоту дискретизации

[4] https://github.com/johnosbb/MicroTFLite

каждого фрейма из 256 сэмплов отдельно, и записывает результат в отдельный двойной буфер с двумя секциями по 128 сэмплов. Еще один двойной буфер хранит результаты вычисления мел-спектрограмм для 30 последовательных фреймов, по 15 для каждой секции. Как только секция n буфера заполняется, предыдущая заполненная секция $n-1$ копируется в holdout-массив, и по окончании текущего фрейма обработки секции $n-1$ и n передаются на следующий этап обработки (вычисление мел-спектрограмм и инференс, соотвтетственно).

Вычисление мал-спектрограмм выполнено с помощью пакета CMSIS-DSP для процессора iMX.RT1062. Процедура вычисления практически полностью совпадает с процедурой на Python (за исключением квантизационного шума, см. ниже).

Инференс модели глубокого обучения происходит раз в примерно четверть секунды, и в среднем занимает чуть меньше 1 миллисекунды, что позволяет обработке и детектированию сигналов происходить без задержек и потерь информации.

Для вывода сообщения о детектировании сигнала на экран используется библиотека U8G2.

Подключение компонентов

Устройство	**Пин устройства**	**Пин Teensy 4.1**	**Примечание**
Цифровой I2S MEMS микрофон (ICS43434)	VDD	3.3V	Питание.
	GND	GND	Земля. Общий провод схемы.
	SCK	Pin 21	Тактовый сигнал I2S.
	SD	Pin 8	Линия данных.
	WS	Pin 20	Выбор канала (Left/Right).
	L/R	GND	Channel Select. GND = Левый, 3.3V = Правый.
I2C Дисплей (SH1106)	VCC	3.3V	Питание.
	GND	GND	Земля. Общий провод схемы.
	SCL	Pin 19	Тактовая линия I2C.
	SDA	Pin 18	Линия данных I2C.

Таблица 1: Схема подключения I2S MEMS микрофона и I2C дисплея к Teensy 4.1

i2S-MEMS микрофон и дисплей с драйвером SH1106 подключены к отладочной плате по схеме, описанной в табл. 1. Дисплей с драйвером SH1106 подключен к пинам 18 (SDA) и 19 (SCL), соответствующим контактам I^2C0 платы. Для корректной работы I^2C линии SCL и SDA необходимо подтянуть к V, для этого были использованы внешние резисторы номиналом 4.7 кОм. Рядом с GND и 3.3V дисплея был подключен керамический контенсатор емкостью 0.1uF.

Демонстрация результатов

Результаты выполнения заданий продемонстрированы в файле course_assignment.mp4. При тестировании собранное устройство показало хорошую способность к детектированию ключевого слова, отсутствие реакции на большую часть окружающих шумов и относительно неплохую устойчивость к использованию слов, сходных по звучанию с кодовым, но отличных от него.

Возможности для улучшения

- **Поддержка различных ключевых слов и нескольких говорящих:** в силу временных и ресурсных ограничений, задача была решена в самом простом варианте, а именно, для фиксированного кодового слова одного фиксированного спикера. Даже в таком варианте модель продемонстрировала некоторую способность к генерализации, поскольку во время тестирования было замечено, что кодовое слово от другого спикера в некоторых случаях успешно детектируется. Однако данный эффект не является намеренным эффектом, и различне обобщения задачи Keyword Spotting в рамках данной работы рассмотрены не были.

- **Исследование алгоритма квантования на предмет возможного улучшения:** просадка метрик после квантования довольно существенная даже с использованием QAT. Возможно, необходимо использовать другие методы QAT/провести более тонкую настройку процесса квантования.

- **Поддержка более точного процесса получения входных данных:** в текущей версии TeensyDuino поддерживается не самая новая версия CMSIS-DSP. Помимо этого, функции из CMSIS-DSP на микроконтроллере и Python дают различные результаты (предположительно из-за квантизационного шума), что не дает обеспечить bit-exact вычисления. Также в силу ограничений Teensy Audio Library сигнал с микрофона сэмплируется на частоте 44100 Гц, и затем ресэмплируется до близкого, но не равного 8 кГц значения. Все это вносит небольшую деградацию в перформанс модели, хотя и не ухудшает перформанс кардинально.

Заключение

В данной работе были продемонстрированы методы создания аппаратно-программной системы для решения задачи распознавания кодового слова. Полученное аппаратное решение продемонстрировало отличные результаты, сравнимые с результатами программной симуляции, что говорит об успешном переносе решения на аппаратную платформу. Несмотря на использование не самой распространенной отладочной платы Teensy 4.1, решение практически полностью опирается на фреймворки, созданные для плат Arduino, и использующие архитектурные решения, ориентированные на простоту использования и отсутствие необходимости глубокого погружения в технические детали имплементации вычислений для предобработки данных и проведения инференса модели глубокого обучения. Данный проект может быть с легкостью адаптирован для решения различных задач, связанных с распознаванием аудиосигнала и обработкой аудио на микроконтроллерах.